

Lab 1: Implementation of Process Scheduling (FCFS, SJF, SRTN, RR, Preemptive Priority)

Theory

Process scheduling is the method used by an operating system to select processes from the ready queue and determine the order in which they are executed by the CPU. Since multiple processes may be ready at the same time but the CPU can execute only a limited number of processes at once, scheduling is required to manage CPU execution efficiently.

A process scheduler selects a process according to a particular scheduling algorithm. Scheduling can be either non-preemptive or preemptive. In non-preemptive scheduling, a running process continues until it finishes or enters a waiting state. In preemptive scheduling, the operating system can interrupt a running process and assign the CPU to another process.

Common measures used to evaluate process scheduling include waiting time, turnaround time, and response time.

$$\text{Waiting Time} = \text{Turnaround Time} - \text{Burst Time}$$

$$\text{Turnaround Time} = \text{Completion Time} - \text{Arrival Time}$$

The main process scheduling algorithms studied in this experiment are FCFS, SJF, SRTN, Round Robin, and Preemptive Priority Scheduling. Each algorithm uses a different criterion for selecting the next process.

Advantages:

- Improves CPU utilization and manages multiple processes efficiently.
- Reduces unnecessary CPU idle time.
- Provides a systematic method for selecting processes.
- Can improve waiting time, response time, or fairness depending on the algorithm.

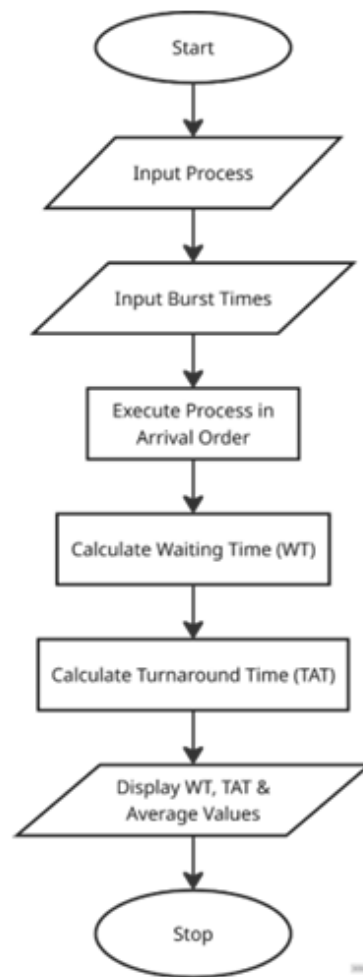
Disadvantages:

- Different algorithms may cause long waiting times for some processes.
- Preemptive algorithms can introduce context-switching overhead.
- Some algorithms may cause starvation of certain processes.
- Choosing an unsuitable scheduling algorithm can reduce overall system performance.

1. First-Come, First-Served (FCFS)

First-Come, First-Served (FCFS) is a non-preemptive process scheduling algorithm in which processes are executed in the order of their arrival in the ready queue. The process that arrives first gets the CPU first, while later processes wait until the earlier processes are completed.

The algorithm uses a simple queue structure. When the CPU becomes available, the process at the front of the queue is selected. Once a process starts execution, it continues until its CPU burst is completed. For example, if three processes P1, P2, and P3 arrive in that order, the execution sequence will generally be $P1 \rightarrow P2 \rightarrow P3$.



Advantages:

- Simple and easy to implement.
- Maintains the order of process arrival.
- Does not require complex scheduling calculations.
- Provides predictable execution order.

Disadvantages:

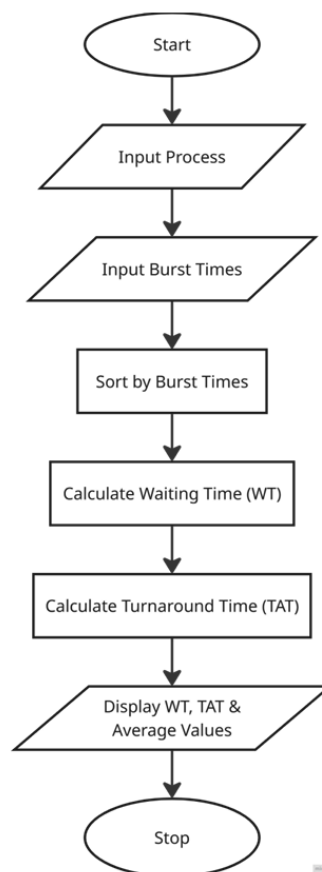
- May result in high average waiting time.
- A long process can make shorter processes wait for a long time.
- Provides poor response time for interactive processes.
- Does not consider burst time or process priority.

2. Shortest Job First (SJF)

Shortest Job First (SJF) is a non-preemptive process scheduling algorithm that selects the process with the smallest CPU burst time from the available processes. The selected process is executed until its burst is completed before another process is selected.

When the CPU becomes available, the scheduler examines the processes that have already arrived and compares their burst times. The process with the shortest burst time is selected. For example, if the available processes have burst times of 8, 4, and 6 units, the process with burst time 4 is selected first.

SJF is designed to reduce the waiting time of processes by executing shorter jobs before longer ones. However, the scheduler needs information about the expected CPU burst time.



Advantages:

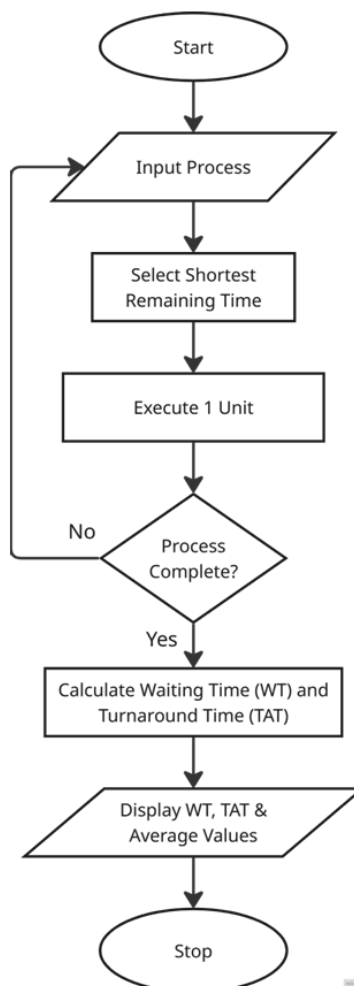
- Can provide low average waiting time.
- Gives faster service to short processes.
- Simple concept and effective when burst times are known.
- Can improve CPU scheduling efficiency.

Disadvantages:

- Requires knowledge or estimation of CPU burst times.
- Long processes may experience high waiting times.
- Starvation can occur if short processes continuously arrive.

3. Shortest Remaining Time Next (SRTN)

Shortest Remaining Time Next (SRTN) is a preemptive process scheduling algorithm that selects the process with the shortest remaining CPU burst time. It is the preemptive version of Shortest Job First (SJF). A currently running process can be interrupted if another available process has a shorter remaining burst time.



Advantages:

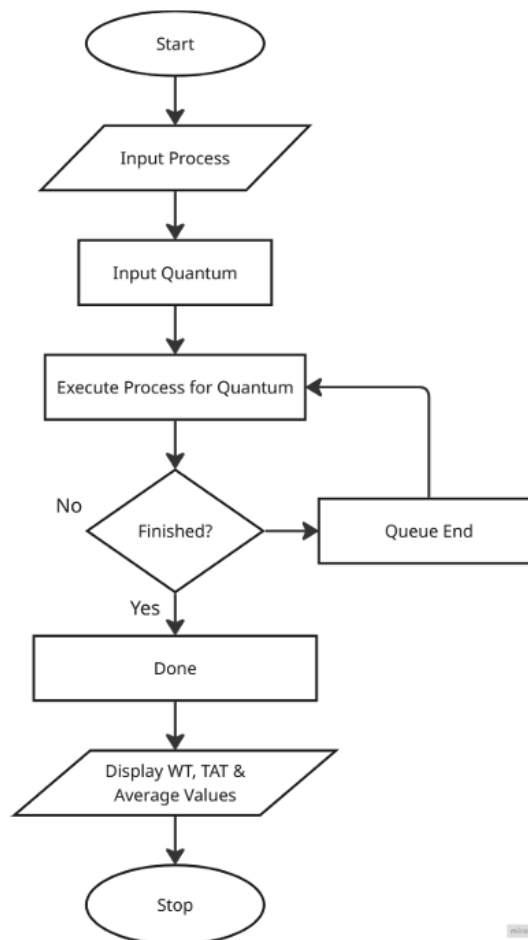
- Can provide low average waiting time.
- Gives preference to processes with shorter remaining execution time.
- Responds to newly arriving short processes quickly.
- Makes better use of preemption when short processes become available.

Disadvantages:

- Requires continuous comparison of remaining burst times.
- Frequent preemption can cause context-switching overhead.
- Longer processes may experience starvation.
- Requires information or estimation of CPU burst times.

4. Round Robin (RR)

Round Robin (RR) is a preemptive process scheduling algorithm in which each ready process is given a fixed amount of CPU time called a time quantum. Processes are executed in a circular order, and a process that does not finish within its time quantum is moved to the back of the ready queue.



Advantages:

- Provides fair CPU allocation among processes.
- Suitable for time-sharing systems.
- Gives good response time for interactive processes.
- Prevents a single process from continuously occupying the CPU.

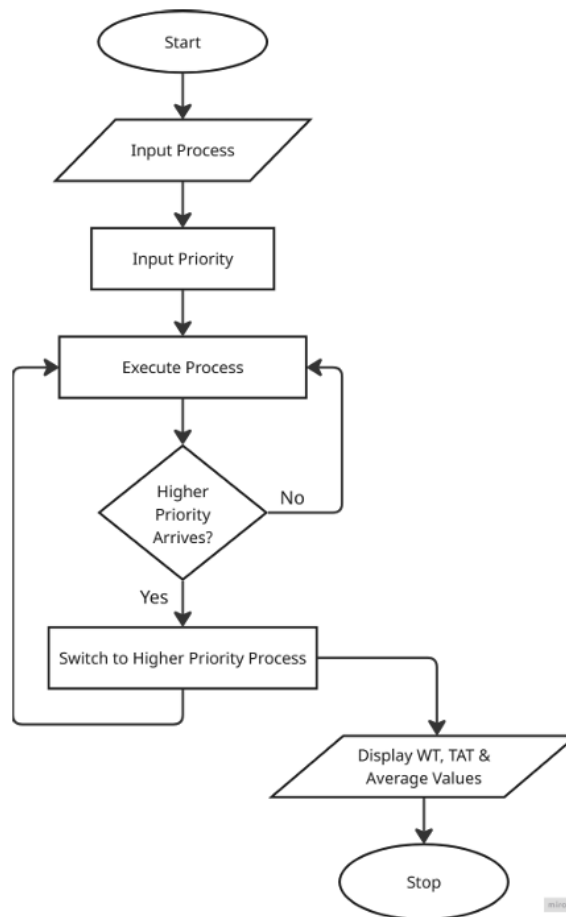
Disadvantages:

- Performance depends heavily on the selected time quantum.
- A very small quantum can cause frequent context switching.
- A very large quantum can make it behave similarly to FCFS.
- May result in higher average waiting time depending on the workload.

5. Preemptive Priority Scheduling

Preemptive Priority Scheduling is a CPU scheduling algorithm in which each process is assigned a priority, and the process with the highest priority among the available processes is selected for execution. In the preemptive version, a running process can be interrupted when a newly arrived process has a higher priority.

When a process arrives, its priority is compared with the priority of the currently executing process. If the new process has a higher priority, the current process is preempted and the new process receives the CPU. The exact meaning of priority values depends on the convention used; commonly, a smaller numerical value represents a higher priority.



Advantages:

- Allows high-priority processes to receive CPU service quickly.
- Useful when some processes are more important than others.
- Supports preemption when a more important process arrives.
- Can provide fast response for high-priority processes.

Disadvantages:

- Low-priority processes may experience starvation.
- Frequent preemption can increase context-switching overhead.
- Performance depends on how priorities are assigned.
- Requires a mechanism such as aging to reduce prolonged waiting of low-priority processes.

Source Code

1. First-Come, First-Served (FCFS)

```
#include<stdio.h>

void findWaitingTime(int processes[], int n, int bt[], int wt[]) {
    wt[0] = 0;
    int i;
    for (i = 1; i < n ; i++ )
        wt[i] = bt[i-1] + wt[i-1] ;
}

void findTurnAroundTime( int processes[], int n, int bt[], int wt[], int tat[]) {
    int i;
    for ( i = 0; i < n ; i++)
        tat[i] = bt[i] + wt[i];
}

void findavgTime( int processes[], int n, int bt[])
{
    int wt[n], tat[n], total_wt = 0, total_tat = 0,i;
    findWaitingTime(processes, n, bt, wt);
    findTurnAroundTime(processes, n, bt, wt, tat);
    printf("Processes  Burst time  Waiting time  Turn around time\n");

    for (i=0; i<n; i++) {
        total_wt = total_wt + wt[i];
        total_tat = total_tat + tat[i];
        printf("  %d\t",i+1));
        printf("\t%d \t", bt[i] );
        printf("\t%d \t",wt[i] );
        printf("\t%d\n",tat[i] );
    }
    float s=(float)total_wt / (float)n;
```

```

float t=(float)total_tat / (float)n;
printf("Average waiting time = %f\n",s);
printf("Average turn around time = %f ",t);
}

```

```

int main() {
    int processes[] = {1, 2, 3};
    int n = sizeof processes / sizeof processes[0];

    int burst_time[] = {10, 5, 8};

    findavgTime(processes, n, burst_time);
    return 0;
}

```

2. Shortest Job First (SJF)

```
#include<stdio.h>
```

```
#include<stdio.h>
```

```

int findNextProcess(int at[], int bt[], int is_completed[], int n, int current_time) {
    int min_index = -1;
    int i;
    for (i = 0; i < n; i++) {
        if (at[i] <= current_time && is_completed[i] == 0) {
            if (min_index == -1 || bt[i] < bt[min_index]) {
                min_index = i;
            }
        }
    }
    return min_index;
}

```

```

void sjf(int pid[], int at[], int bt[], int wt[], int tat[], int n) {
    int completed = 0, current_time = 0;

```

```

int is_completed[n],i;

for (i = 0; i < n; i++)
    is_completed[i] = 0;

while (completed != n) {
    int index = findNextProcess(at, bt, is_completed, n, current_time);
    if (index == -1) {
        current_time++;
    } else {
        wt[index] = current_time - at[index];
        current_time += bt[index];
        tat[index] = wt[index] + bt[index];

        is_completed[index] = 1;
        completed++;
    }
}

}

void print(int pid[], int at[], int bt[], int wt[], int tat[], int n) {
    float avg_wt = 0, avg_tat = 0;
    int i;
    printf("Process Scheduling: SJF\n");
    printf("\nPID\tAT\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\n",pid[i], at[i], bt[i], wt[i], tat[i]);
        avg_wt += wt[i];
        avg_tat += tat[i];
    }
    printf("Average waiting time= %f\n",avg_wt/n);
    printf("Average turn around time= %f",avg_tat/n);
}

```

```

int main() {
    int n = 3;

    int pid[3] = {1, 2, 3};
    int at[3] = {0, 3, 2};
    int bt[3] = {3, 5, 4};

    int wt[3], tat[3], ft[3];

    sjf(pid, at, bt, wt, tat, n);

    print(pid, at, bt, wt, tat, n);

    return 0;
}

```

3. Shortest Remaining Time Next (SRTN)

```

#include <stdio.h>

int findNextProcess(int at[], int rt[], int is_completed[], int n, int current_time) {
    int min_index = -1;
    int i;

    for (i = 0; i < n; i++) {
        if (at[i] <= current_time && is_completed[i] == 0) {
            if (min_index == -1 || rt[i] < rt[min_index]) {
                min_index = i;
            }
        }
    }

    return min_index;
}

```

```

void srtn(int pid[], int at[], int bt[], int wt[], int tat[], int rt[], int n) {

    int completed = 0, current_time = 0, i;
    int is_completed[n];

    for (i = 0; i < n; i++)
        is_completed[i] = 0;

    while (completed != n) {

        int index = findNextProcess(at, rt, is_completed, n, current_time);

        if (index == -1) {
            current_time++;
        } else {

            rt[index]--;
            current_time++;

            if (rt[index] == 0) {
                tat[index] = current_time - at[index];
                wt[index] = tat[index] - bt[index];

                is_completed[index] = 1;
                completed++;
            }
        }
    }
}

void print(int pid[], int at[], int bt[], int wt[], int tat[], int n) {

    float avg_wt = 0, avg_tat = 0;

```

```

int i;

printf("\nProcess Scheduling: SRTF (SRTN)\n");
printf("\nPID\tAT\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i], wt[i], tat[i]);

    avg_wt += wt[i];
    avg_tat += tat[i];
}
printf("\nAverage Waiting Time = %.2f\n", avg_wt / n);
printf("Average Turnaround Time = %.2f\n", avg_tat / n);
}

int main() {
    int n = 3;
    int i;
    int pid[3] = {1, 2, 3};
    int at[3] = {0, 3, 2};
    int bt[3] = {3, 5, 4};
    int wt[3], tat[3], rt[3];

    for (i = 0; i < n; i++)
        rt[i] = bt[i];

    srtn(pid, at, bt, wt, tat, rt, n);
    print(pid, at, bt, wt, tat, n);

    return 0;
}

```

4. Round Robin (RR)

```
#include <stdio.h>

void roundRobin(int pid[], int at[], int bt[], int n, int quantum) {
    int rem_bt[n], wt[n], tat[n], ct[n], i, j ;
    for(i=0; i<n; i++)
        rem_bt[i] = bt[i];

    int queue[100], front = 0, rear = 0;
    int visited[n];

    for( i=0; i<n; i++)
        visited[i] = 0;

    int time = 0, completed = 0;

    for( i=0; i<n; i++) {
        if(at[i] == 0) {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }

    while(completed < n) {

        if(front == rear) {
            time++;

            for( i=0; i<n; i++) {
                if(!visited[i] && at[i] <= time) {
                    queue[rear++] = i;
                    visited[i] = 1;
                }
            }
        }
    }
}
```

```

    }
    continue;
}

int i = queue[front++];

if(rem_bt[i] > quantum) {
    time += quantum;
    rem_bt[i] -= quantum;
}
else {
    time += rem_bt[i];
    rem_bt[i] = 0;
    ct[i] = time;
    completed++;
}

for( j=0; j<n; j++) {
    if(!visited[j] && at[j] <= time) {
        queue[rear++] = j;
        visited[j] = 1;
    }
}

if(rem_bt[i] > 0)
    queue[rear++] = i;
}

float total_wt = 0, total_tat = 0;

for(i=0; i<n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
}

```

```

        total_wt += wt[i];
        total_tat += tat[i];
    }

    printf("\nPID\tAT\tBT\tCT\tWT\tTAT\n");
    printf("-----\n");

    for(i=0; i<n; i++)
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",pid[i], at[i], bt[i], ct[i], wt[i], tat[i]);

    printf("\nAverage Waiting Time = %.2f", total_wt/n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat/n);
}

int main() {
    int pid[] = {1,2,3};
    int at[] = {0,3,5};
    int bt[] = {3,5,4};
    int quantum;

    printf("Enter Time Quantum: ");
    scanf("%d",&quantum);

    roundRobin(pid, at, bt, 3, quantum);

    return 0;
}

```

5. Preemptive Priority Scheduling

```
#include <stdio.h>

void preemptivePriority(int pid[], int at[], int bt[], int pr[], int n) {
    int rem_bt[20], ct[20], wt[20], tat[20];
    int i, highest;
    int completed = 0;
    int time = 0;
    float total_wt = 0, total_tat = 0;

    for(i = 0; i < n; i++)
        rem_bt[i] = bt[i];

    while(completed < n) {
        highest = -1;
        for(i = 0; i < n; i++) {
            if(at[i] <= time && rem_bt[i] > 0) {
                if(highest == -1 || pr[i] < pr[highest])
                    highest = i;
            }
        }

        if(highest == -1) {
            time++;
            continue;
        }

        rem_bt[highest]--;
        time++;

        if(rem_bt[highest] == 0) {
            ct[highest] = time;
            completed++;
        }
    }
}
```

```

    }
}

for(i = 0; i < n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    total_wt += wt[i];
    total_tat += tat[i];
}

printf("\nPID\tAT\tBT\tPR\tCT\tWT\tTAT\n");
printf("-----\n");
for(i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",pid[i], at[i], bt[i], pr[i], ct[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f", total_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);
}

int main() {
    int n = 4;
    int pid[] = {1, 2, 3, 4};
    int at[] = {0, 1, 2, 3};
    int bt[] = {4, 3, 1, 2};

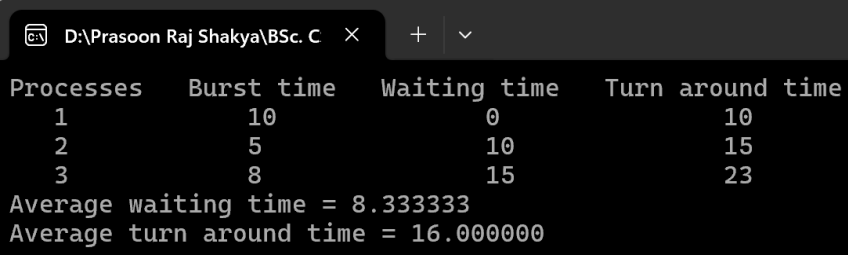
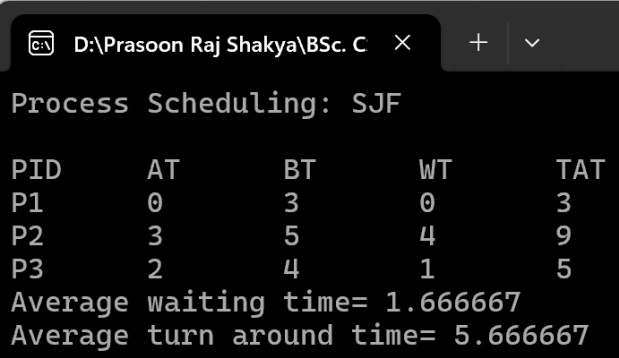
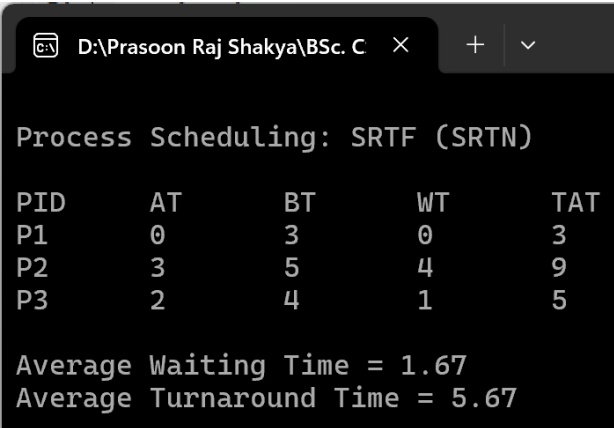
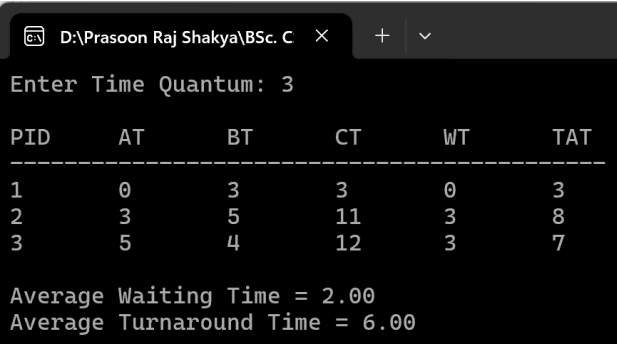
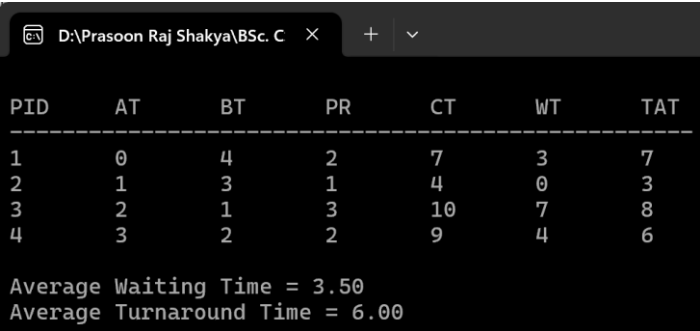
    /* Smaller value = Higher Priority */
    int pr[] = {2, 1, 3, 2};

    preemptivePriority(pid, at, bt, pr, n);

    return 0;
}

```

Output

FCFS	 <pre> D:\Prasoon Raj Shakya\BSc. C X + v Processes Burst time Waiting time Turn around time 1 10 0 10 2 5 10 15 3 8 15 23 Average waiting time = 8.333333 Average turn around time = 16.000000 </pre>
SJF	 <pre> D:\Prasoon Raj Shakya\BSc. C X + v Process Scheduling: SJF PID AT BT WT TAT P1 0 3 0 3 P2 3 5 4 9 P3 2 4 1 5 Average waiting time= 1.666667 Average turn around time= 5.666667 </pre>
SRTN	 <pre> D:\Prasoon Raj Shakya\BSc. C X + v Process Scheduling: SRTF (SRTN) PID AT BT WT TAT P1 0 3 0 3 P2 3 5 4 9 P3 2 4 1 5 Average Waiting Time = 1.67 Average Turnaround Time = 5.67 </pre>
RR	 <pre> D:\Prasoon Raj Shakya\BSc. C X + v Enter Time Quantum: 3 PID AT BT CT WT TAT ----- 1 0 3 3 0 3 2 3 5 11 3 8 3 5 4 12 3 7 Average Waiting Time = 2.00 Average Turnaround Time = 6.00 </pre>
Priority	 <pre> D:\Prasoon Raj Shakya\BSc. C X + v PID AT BT PR CT WT TAT ----- 1 0 4 2 7 3 7 2 1 3 1 4 0 3 3 2 1 3 10 7 8 4 3 2 2 9 4 6 Average Waiting Time = 3.50 Average Turnaround Time = 6.00 </pre>

Conclusion

In this lab, FCFS, SJF, SRTN, Round Robin, and Preemptive Priority Scheduling algorithms were studied and implemented in C. The implementation helped in understanding how an operating system selects processes from the ready queue and allocates CPU time using different scheduling techniques. It also provided practical understanding of waiting time, turnaround time, completion time, and the execution order of processes.

The lab demonstrated the differences between non-preemptive and preemptive scheduling methods. It was learned that each algorithm uses a different criterion for selecting the next process, such as arrival order, burst time, remaining time, time quantum, or priority. Overall, the experiment provided a practical understanding of how process scheduling algorithms manage CPU resources and affect the execution of processes.